COURSE SLIDES · WSQ

SQL Fundamental for Beginners

WSQ Course Code: TGS-2020505790

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Trainer: Dr. Alfred Ang · www.tertiarycourses.com.sg · enquiry@tertiaryinfotech.com · +65 6100 0613

Version v14 · 11 August 2026

COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

1

Three times a day

Take the AM, PM and Assessment digital attendance — mandatory for WSQ-funded courses.

2

QR code from SSG

The trainer or administrator displays the attendance QR code from the SSG portal.

3

Scan and submit

Scan the QR code with your mobile phone camera and submit your attendance.

4

75% minimum

A minimum 75% attendance rate is required to be eligible for assessment and funding.

About the Trainer

Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

YOUR TRAINER

About the Trainer

Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE

Principal Trainer & Founder, Tertiary Infotech Academy Pte Ltd

QUALIFICATIONS

PhD — over 20 years of training and consulting experience in data, analytics and software engineering.

DELIVERS

WSQ courses on SQL, databases, data engineering, analytics and AI.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

ICE-BREAKER

Let's Know Each Other

1

Your name, organisation and role

2

Your experience with spreadsheets, databases or programming

3

The data you want to query and analyse after this course

HOUSEKEEPING

Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. 75% attendance is required.

LMS / TMS · lms-tms.tertiaryinfotech.com

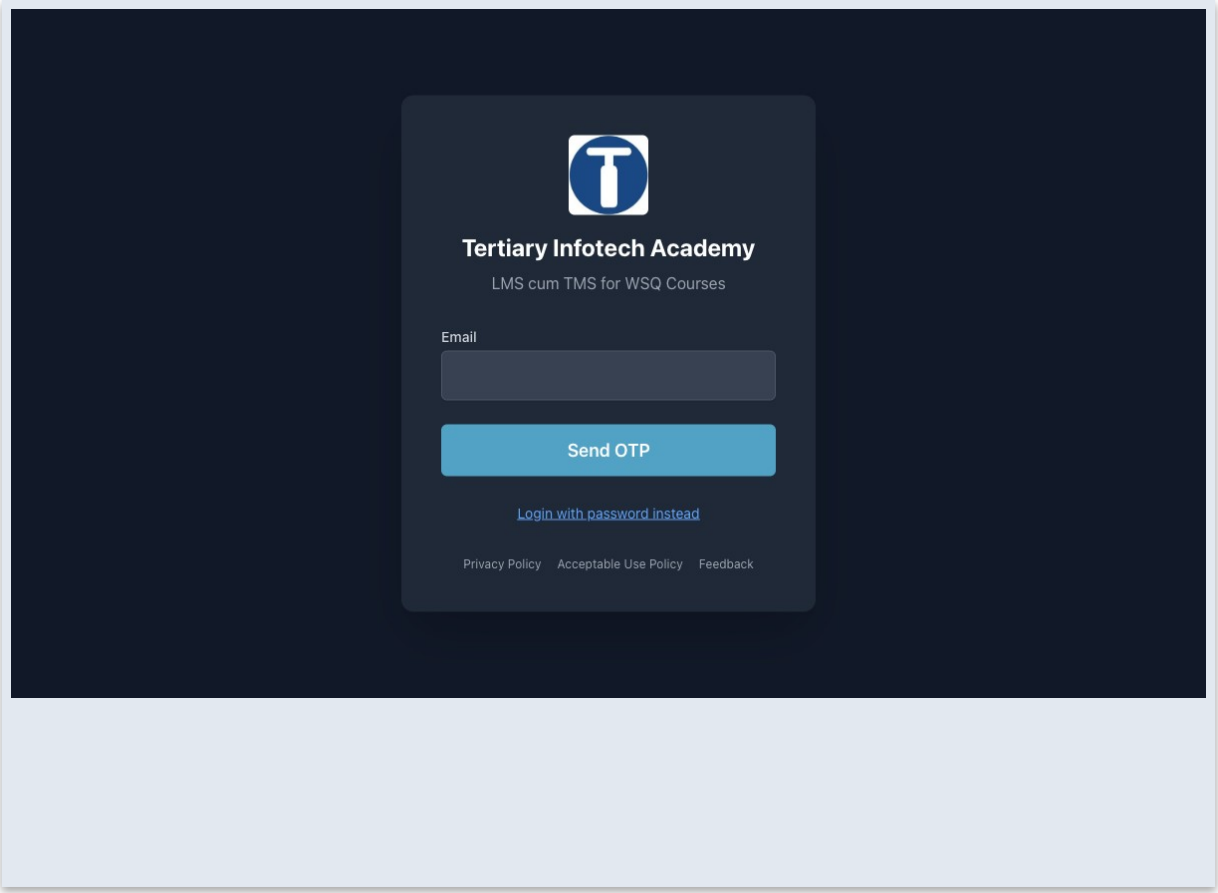
Download Course Material

1. Log in to the LMS portal: lms-tms.tertiaryinfotech.com

2. Open My Courses and select SQL Fundamental for Beginners.

3. Open the Course Materials section of the course.

4. Download the slides, Learner Guide and the datasets folder (mock data).

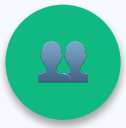


ONE BUSINESS STORY, ELEVEN LABS

The Course Data — SG Mart Pte Ltd

8 Outlets

Orchard, Tampines, Jurong East, Woodlands, Bedok, Punggol, Clementi, Serangoon.



42 Staff

Managers, supervisors and cashiers — with roles, salaries and hire dates.

60 Members

Loyalty tiers from Basic to Platinum, with points balances.

25 Products

Fresh, chilled, bakery, beverages, snacks, household, personal care, frozen.

180 Orders

A full year of 2025 trading across every outlet and channel.

685 Order Lines

The fact table — quantity, price and discount on every item sold.

EVERY LAB HAS ITS OWN DATASET FOLDER

How to Load the Data

Run the seed script

- Open seed_sqlite.sql from the lab folder.
- Execute the whole script (▶ / F9).
- Creates and fills every table in one step.
- Fastest way to start a lab.

Import the CSV / Excel

- Each lab folder holds its own CSVs + SQL.
- SQLite Studio: Tools → Import.
- Tick 'First line represents column names'.
- The same files open directly in Excel.

Use the prebuilt database

- labs/_all/sgmart.db — every table, ready.
- Database → Add a database, then Connect.
- No import step at all.
- Also includes the Excel workbook.

SCHEDULE

Lesson Plan — 1 Day, 8 Training Hours

Morning

- Morning (9:30 am – 1:30 pm)
  - Welcome, introductions & digital attendance (AM)
  - Topic 1: Data Modeling — Labs 1–3
  - Topic 2: Data Processing & Analysis — Labs 4–6
  - Lunch break (1:30 – 2:30 pm)

Afternoon

- Afternoon (2:30 pm – 6:30 pm)
  - Topic 3: Data Transformation — Labs 7–9
  - Topic 4: Introduction to Data Warehouse — Labs 10–11
  - Course feedback (TRAQOM) & Assessment digital attendance
  - Briefing, then Final Assessment: PP (70 min) + OQ (20 min)

Skills Framework

- TSC Title: Data Engineering
  - TSC Code: ICT-DIT-3005-1.1
  - Framework: ICT Skills Framework
- This course develops the TSC abilities:
  - A1: Identify relevant data sources, processes and relationships in accordance to business requirements
  - A2: Propose methods and tools to gather and process data, and minimise confounding variables and data limitations
  - A3: Apply data analysis and data profiling to improve the clarity, quality and integrity of valid data
  - A4: Process multiple streams of data using data systems
  - A5: Utilise data systems and platform capabilities to solve new data problems

Skills Framework — TSC Abilities (continued)

- A6: Transform data to meet specified business requirements
- A7: Operate data warehouse systems to balance optimisation of data access with batch loading and resource utilisation
- A8: Perform data mapping from source to destination systems
- A9: Resolve data issues arising from data processing activities

• Knowledge items K1–K7 (data sources, modeling, warehousing, queries, transformation, mapping) are covered across Topics 1–4.

WHAT YOU'LL ACHIEVE

Learning Outcomes

1

LO1 · Data Modeling

Apply data modeling for business processes.

2

LO2 · Data Processing

Apply data processing and analysis using SQL.

3

LO3 · Data Transformation

Apply data transformation from multiple data sources.

4

LO4 · Data Warehouse

Apply data mapping to data warehouse.

4 TOPICS · 11 HANDS-ON LABS

Course Outline

1

Topic 1 · Data Modeling (A1, A2, K1, K2)

Data Sources · Data Modeling · SQL Data Types · Create & Manage Databases

2

Topic 2 · Data Processing and Analysis (A3, A5, K4)

SQL Queries · SQL Operators · Insert, Update & Delete

3

Topic 3 · Data Transformation (A4, A6, K5, K6)

Aggregate Data · Join Data · Group Data

4

Topic 4 · Introduction to Data Warehouse (A7, A8, A9, K3, K7)

Overview of Data Warehouse · Relational Databases · Stored Procedures

BEFORE YOU BEGIN

Briefing for Assessment

1. Place phones and other materials under the table or on the floor.
2. No photos or recording of assessment scripts.
3. No discussion during the assessment.
4. Use a black or blue pen for hard-copy assessments.
5. No liquid paper or correction tape.
6. Assessment scripts are collected when the time is up.

ASSESSMENT · 90 MINUTES TOTAL

Final Assessment

1

Practical Performance (PP)

Hands-on SQL tasks in SQLite Studio · 70 minutes · assessor-to-candidate ratio 1:10.

2

Oral Questioning (OQ)

Spoken knowledge questions · 20 minutes · one-to-one with the assessor.

3

Open book

Course slides, Learner Guide and approved materials only.

4

To be funded

75% minimum attendance and a result of 'Competent'. An appeal process is available.

ON ASSESSMENT DAY

Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Practical Performance (PP) — 70 min, open book
4. Oral Questioning (OQ) — 20 min, 1-to-1
5. Submit on the LMS — upload your answers
6. Sign the Assessment Summary Record

SQL FOUNDATIONS

What is SQL?

OVERVIEW

What is SQL?

1

Structured Query Language

The standard language for storing, manipulating and retrieving data in databases.

2

An open standard

ANSI standard since 1986, ISO standard since 1987.

3

Works everywhere

MySQL, SQL Server, Access, Oracle, Sybase, Informix, PostgreSQL and more.

4

Declarative

You state WHAT data you want; the database engine works out how to get it.

CAPABILITIES

What Can SQL Do?

1

Query

Execute queries and retrieve data from a database.

2

Change data

Insert, update and delete records.

3

Define structure

Create new databases, tables, views and indexes.

4

Automate

Create stored procedures for reusable server-side logic.

5

Secure

Set permissions on tables, procedures and views.

6

Analyse

Aggregate, join and group data for analysis.

MILESTONES

A Short History of SQL

1. 1970 — E.F. Codd publishes the relational model
2. 1975–77 — first SQL implementations & IBM System R
3. 1980s — commercial releases: Oracle, SQL/DS, DB2
4. 1986/87 — SQL becomes the ANSI & ISO standard
5. By 1992 — over 100 SQL products on the market

ANSI-COMPLIANT ENGINES · SELECT · UPDATE · DELETE · INSERT · WHERE

Databases That Speak SQL

1

MySQL / MariaDB

The web's workhorse open-source databases.

2

PostgreSQL

Advanced open-source object-relational database.

3

SQLite

Zero-config embedded database — our classroom engine.

4

Microsoft SQL Server

Enterprise database with T-SQL and stored procedures.

5

Oracle

Enterprise market leader since 1980.

6

DB2 · Sybase · Access

IBM's DB2, SAP's Sybase and desktop MS Access.

Components of a Table

Rows (records)

- A database is one or more tables
  - Each table is a collection of rows
  - Every row of a table has the same row type
  - A row is the smallest unit inserted or deleted

Columns (fields)

- Columns define the fields
  - The n-th field of every row is the n-th column
  - Each column has a name and a data type
  - Rows × columns = the relational grid

CLASSROOM TOOLS

Your Workbench — SQLite Studio

1

Desktop app

Browse and edit SQLite database files visually — created by Paweł Salawa.

2

Open source

Free, GPLv3-licensed, cross-platform (C++/Qt) — download at sqlitestudio.pl.

3

SQL editor built in

Write and run every query in this course from Tools → Open SQL editor.

4

Cloud alternative

No install? Use the free sqliteonline.com in your browser.

01

TOPIC 01

Data Modeling

Data Sources · Data Modeling · SQL Data Types · Create & Manage Databases

TSC MAPPING · A1, A2, K1, K2

Key Concepts — Data Modeling

1

Relational databases

A database is one or more tables; each table is rows (records) and columns (fields) with a fixed row type.

2

DDL statements

CREATE / ALTER / DROP define databases, tables, indexes and their columns and data types.

3

Constraints

NOT NULL, DEFAULT, UNIQUE, CHECK, PRIMARY KEY and FOREIGN KEY enforce rules on the data.

4

Data modeling

Primary and foreign keys link tables into entity relationships that mirror the business process.

CREATE DATABASE Statement

- The CREATE DATABASE statement creates a new SQL database.
- DROP DATABASE permanently deletes an existing database — use with care.

SYNTAX

```
CREATE DATABASE databasename;

DROP DATABASE databasename;
```

EXAMPLE

```
CREATE DATABASE testDB;

DROP DATABASE testDB;
```

CREATE TABLE Statement

- CREATE TABLE creates a new table in a database.
- Each column is declared with a name and a data type.

SYNTAX

```
CREATE TABLE table_name (
  column1 datatype,
  column2 datatype,
  column3 datatype
);
```

EXAMPLE

```
CREATE TABLE Persons (
  PersonID int,
  LastName varchar(255),
  FirstName varchar(255),
  City varchar(255)
);
```

String Data Types

1

CHAR(n)

Fixed-length string, padded to n characters (0–255).

2

VARCHAR(n)

Variable-length string up to n characters — the everyday text type.

3

TEXT

Long free text (articles, descriptions) up to 65,535 bytes.

4

BLOB

Binary large object — images, files and other raw bytes.

5

ENUM

One value chosen from a fixed list you define.

6

SET

Zero or more values chosen from a fixed list.

Numeric Data Types

1

INT / INTEGER

Whole numbers — IDs, counts, populations.

2

SMALLINT / BIGINT

Smaller or larger integer ranges to fit the data.

3

DECIMAL(p,s)

Exact fixed-point numbers — money and precise quantities.

4

FLOAT / REAL / DOUBLE

Approximate floating-point numbers for scientific values.

5

BOOLEAN

True/false flags (stored as 0 / 1).

6

BIT

Individual bit values for compact flags.

Date and Time Data Types

1

DATE

Calendar date: YYYY-MM-DD.

2

TIME

Time of day: hh:mm:ss.

3

DATETIME

Date and time combined: YYYY-MM-DD hh:mm:ss.

4

TIMESTAMP

Date-time stored relative to epoch — auto-updates on change.

5

YEAR

Four-digit year.

6

Note

SQLite stores dates as TEXT, REAL or INTEGER under the hood.

ALTER TABLE Statement

- ALTER TABLE adds, deletes or modifies columns in an existing table.
- It is also used to add or drop constraints after creation.

SYNTAX

```
ALTER TABLE table_name
ADD column_name datatype;
```

EXAMPLE

```
ALTER TABLE Customers
ADD Email varchar(255);
```

SQL Constraints

1

NOT NULL

Column must always contain a value.

2

UNIQUE

All values in the column must be different.

3

PRIMARY KEY

NOT NULL + UNIQUE — uniquely identifies each row; one per table.

4

FOREIGN KEY

Links a column to the primary key of another table.

5

CHECK

Limits the values allowed in a column by a condition.

6

DEFAULT

Supplies a value automatically when none is given.

NOT NULL & DEFAULT Constraints

- NOT NULL forces a column to always hold a value.
- DEFAULT fills in a value automatically when the INSERT omits one.

SYNTAX

```
CREATE TABLE t (
  col datatype NOT NULL,
  col2 datatype DEFAULT 'value'
);
```

EXAMPLE

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  City varchar(255) DEFAULT 'Sandnes'
);
```

UNIQUE & CHECK Constraints

- UNIQUE guarantees no duplicate values in a column — a table may have many UNIQUE constraints.
- CHECK limits the range of values a column accepts.

SYNTAX

```
CREATE TABLE t (
  col int NOT NULL UNIQUE,
  age int CHECK (age >= 18)
);
```

EXAMPLE

```
CREATE TABLE Persons (
  ID int NOT NULL UNIQUE,
  Age int CHECK (Age >= 18)
);
```

PRIMARY KEY Constraint

- The PRIMARY KEY uniquely identifies each record — UNIQUE and NOT NULL combined.
- A table has exactly ONE primary key; it may span one or several columns.

SYNTAX

```
CREATE TABLE t (
  ID int NOT NULL,
  ...,
  PRIMARY KEY (ID)
);
```

EXAMPLE

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  PRIMARY KEY (ID)
);
```

FOREIGN KEY Constraint

- A FOREIGN KEY links two tables: a field in the child table refers to the PRIMARY KEY of the parent table.
- This is the mechanism that turns separate tables into a data model.

SYNTAX

```
CREATE TABLE child (
  ...,
  FOREIGN KEY (col)
  REFERENCES parent(pk_col)
);
```

EXAMPLE

```
CREATE TABLE Orders (
  OrderID int NOT NULL,
  PersonID int,
  PRIMARY KEY (OrderID),
  FOREIGN KEY (PersonID)
  REFERENCES Persons(PersonID)
);
```

AUTO INCREMENT Field

- Auto-increment generates a unique number automatically for each new record.
- Usually applied to the primary key. In SQLite an INTEGER PRIMARY KEY auto-increments by default.

SYNTAX

```
CREATE TABLE t (
  id int NOT NULL AUTO_INCREMENT,
  ...,
  PRIMARY KEY (id)
);
```

EXAMPLE

```
CREATE TABLE Persons (
  Personid int NOT NULL AUTO_INCREMENT,
  LastName varchar(255) NOT NULL,
  PRIMARY KEY (Personid)
);
```

CREATE INDEX Statement

- Indexes speed up searches and queries — the database finds rows without scanning the whole table.
- Users never see the index; it works silently behind SELECT and WHERE.

SYNTAX

```
CREATE INDEX index_name
ON table_name (column1, column2);
```

EXAMPLE

```
CREATE INDEX idx_lastname
ON Persons (LastName);
```

TOPIC 01

Data Modeling with Keys

What data modeling is

- Explores data-oriented structures for the business
  - Identifies entity types (like classes) and their attributes
  - Spans conceptual → logical → physical models
  - Mirrors real business processes and relationships

How keys create the model

- PRIMARY KEY uniquely identifies each entity row
  - FOREIGN KEY points a child row at its parent
  - One-to-many: one Country has many Cities
  - Relationships form the entity-relationship diagram

SECURITY AWARENESS

Never paste user input straight into SQL.

SQL injection places malicious code in statements via web input — one of the most common hacking techniques. Always validate input and use parameterised queries.

WHAT YOU'LL DO

Hands-On Labs — Data Modeling

Lab 1

- Set Up SQLite Studio & Import the SG Mart Database

Lab 2

- Create a Database and Tables

Lab 3

- Model Data with Constraints and Keys

TOPIC 01 · HANDS-ON

Set Up SQLite Studio & Import the SG Mart Database

LAB 1

Install SQLite Studio, connect it to the SG Mart sample database and explore its tables — your data source for the whole course.

You'll build

A working SQLite Studio with the SG Mart database connected, showing the Outlets, Products and Categories tables.

Tools: SQLite Studio, sgmart.db mock database, LMS course materials

Lab 1 — How It Flows

1. Download & install SQLite Studio
2. Get sgmart.db from the LMS
3. Add & connect the database
4. Browse the tables & open a CSV in Excel

LAB 1 · VERIFY

Set Up SQLite Studio & Import the SG Mart Database

✅ Test it

The Databases panel shows the connected sgmart database with the Outlets, Products, Customers, Orders and OrderItems tables; the Data tab displays outlet rows; and the ORDER BY query returns 8 outlets led by SG Mart Tampines Hub (1680.5 sqm).

TOPIC 01 · HANDS-ON

Create a Database and Tables

LAB 2

Create your own database named 'sgmart_practice' and build the Outlets, Categories, Suppliers and Products tables with CREATE TABLE, choosing an appropriate data type for every column.

You'll build

A new 'sgmart_practice' database containing four tables whose columns, data types and defaults model the SG Mart retail business.

Tools: SQLite Studio, SQL Editor, lab-02 dataset (CSV + seed script)

Lab 2 — How It Flows

1. Create the 'sgmart_practice' database
2. Open the SQL Editor
3. CREATE TABLE Outlets / Categories / Suppliers / Products
4. Load the seed data & inspect the schema

LAB 2 · VERIFY

Create a Database and Tables

✅ Test it

The sgmart_practice database lists Outlets, Categories, Suppliers and Products; each Structure tab shows the declared columns, data types, defaults and primary keys; and the counts return 8, 8, 9 and 25 rows.

TOPIC 01 · HANDS-ON

Model Data with Constraints and Keys

LAB 3

Build a Persons–PersonOrders mini-model: enforce data rules with NOT NULL, UNIQUE, DEFAULT and CHECK, link the tables with a PRIMARY KEY / FOREIGN KEY relationship, and speed up searches with an index.

You'll build

A two-table data model (Persons ← PersonOrders) with working constraints, a foreign-key relationship and an index.

Tools: SQLite Studio, SQL Editor, lab-03 dataset (Persons + PersonOrders)

Lab 3 — How It Flows

1. CREATE Persons with constraints
2. Test the constraints with INSERTs
3. CREATE PersonOrders with a FOREIGN KEY
4. ALTER TABLE & CREATE INDEX

LAB 3 · VERIFY

Model Data with Constraints and Keys

✅ Test it

The valid insert appears in Persons with City = 'Singapore'; the three rule-breaking inserts each raise a constraint error; the join returns 7 matched orders led by 312.75; PersonOrders shows a foreign key to Persons in its DDL; and idx_lastname is listed under Indexes.

TOPIC RECAP

Recap — Data Modeling

- You can now: LO1 — identify relevant data sources and set up the SQL workbench (A1, A2)
- You can now: LO1 — create and manage databases and tables with SQL DDL (A1, A2)
- You can now: LO1 — apply data modeling for business processes with constraints, keys and relationships (A1, A2)
- Full step-by-step instructions for every lab are in the Learner Guide.

Tea Break

15 minutes

02

TOPIC 02

Data Processing and Analysis

SQL Queries · SQL Operators · Insert, Update & Delete

TSC MAPPING · A3, A5, K4

Key Concepts — Data Processing and Analysis

1

Queries

SELECT retrieves data into a result-set; DISTINCT removes duplicates; WHERE filters rows.

2

Operators

AND/OR/NOT, IN, BETWEEN, LIKE wildcards and IS NULL build precise filter conditions.

3

Shaping results

ORDER BY sorts, LIMIT caps the row count, and aliases (AS) rename columns and tables.

4

Changing data

INSERT INTO adds records, UPDATE modifies them and DELETE removes them — always with WHERE.

Key Elements of SQL

1

Queries

Retrieve data against some criteria — the SELECT family.

2

Statements

Control transactions, sessions, connections and program flow.

3

Clauses

Components of queries and statements — WHERE, GROUP BY, ORDER BY.

4

Expressions

Combinations of symbols and operators producing values.

5

Predicates

Conditions that evaluate true or false to filter rows.

6

Result-set

The table of rows a query returns.

SELECT Statement

- SELECT retrieves data from a database into a result-set.
- Use * for all columns, or name the columns you need.

SYNTAX

```
SELECT column1, column2
FROM table_name;

SELECT * FROM table_name;
```

EXAMPLE

```
SELECT * FROM City;

SELECT first_name, last_name
FROM people;
```

SELECT DISTINCT Statement

- SELECT DISTINCT returns only the different (distinct) values.
- Use it when a column holds many duplicates and you want each value once.

SYNTAX

```
SELECT DISTINCT column1, column2
FROM table_name;
```

EXAMPLE

```
SELECT DISTINCT CountryCode
FROM City;
```

WHERE Clause

- WHERE filters records — only rows that satisfy the condition are returned.
- Works with =, <, >, <=, >=, <> and the SQL operators.

SYNTAX

```
SELECT column1, column2
FROM table_name
WHERE condition;
```

EXAMPLE

```
SELECT * FROM City
WHERE District = 'Kabol';
```

AND · OR · NOT Operators

1

AND

Row shown only if ALL conditions are true.

2

OR

Row shown if ANY condition is true.

3

NOT

Row shown if the condition is NOT true.

4

Combine freely

Group with parentheses: WHERE a AND (b OR c).

5

Example

WHERE Country='Germany' AND City='Berlin'

6

Example

WHERE NOT Country='UK'

IN & BETWEEN Operators

- IN matches any value in a list — shorthand for multiple ORs.
- BETWEEN selects values inside an inclusive range of numbers, text or dates.

SYNTAX

```
WHERE column IN (v1, v2, ...);

WHERE column BETWEEN v1 AND v2;
```

EXAMPLE

```
SELECT * FROM City
WHERE CountryCode IN ('AFG','NLD');

SELECT * FROM Products
WHERE Price BETWEEN 10 AND 20;
```

LIKE Operator & Wildcards

- LIKE searches for a pattern in a column.
- % matches zero or more characters; _ matches exactly one character.

SYNTAX

```
SELECT columns FROM table
WHERE column LIKE pattern;
```

EXAMPLE

```
-- names starting with 'a'
SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';

-- countries containing 'island'
SELECT Name FROM Country
WHERE Name LIKE '%island%';
```

NULL Values & EXISTS

- A NULL field has no value — test it with IS NULL / IS NOT NULL, never with =.
- EXISTS returns true if a subquery returns one or more records.

SYNTAX

```
WHERE column IS NULL;

WHERE EXISTS
  (SELECT col FROM t WHERE cond);
```

EXAMPLE

```
SELECT CustomerName FROM Customers
WHERE Address IS NULL;

SELECT SupplierName FROM Suppliers s
WHERE EXISTS (SELECT 1 FROM Products
  WHERE SupplierID = s.SupplierID
  AND Price < 20);
```

ORDER BY & LIMIT

- ORDER BY sorts the result-set — ascending by default, DESC for descending.
- LIMIT caps how many records are returned.

SYNTAX

```
SELECT columns FROM table
ORDER BY col1 ASC|DESC
LIMIT number;
```

EXAMPLE

```
-- top 10 cities by population
SELECT * FROM City
ORDER BY Population DESC
LIMIT 10;
```

Aliases (AS)

- Aliases give a table or column a temporary, more readable name.
- The alias exists only for the duration of that query.

SYNTAX

```
SELECT column AS alias_name
FROM table_name AS t;
```

EXAMPLE

```
SELECT CountryCode AS CC
FROM City;

SELECT CustomerID AS ID,
       CustomerName AS Customer
FROM Customers;
```

INSERT INTO Statement

- INSERT INTO adds new records to a table.
- Name the columns you supply; omitted columns take their DEFAULT (or NULL).

SYNTAX

```
INSERT INTO table_name
  (column1, column2, ...)
VALUES (value1, value2, ...);
```

EXAMPLE

```
INSERT INTO City (Name, CountryCode)
VALUES ('Singapore', 'SGP');
```

UPDATE Statement

- UPDATE modifies existing records.
- ALWAYS include a WHERE clause — without it every row is updated.

SYNTAX

```
UPDATE table_name
SET column1 = value1,
    column2 = value2
WHERE condition;
```

EXAMPLE

```
UPDATE City
SET Name = 'Kabol'
WHERE District = 'Kabol';
```

DELETE Statement

- DELETE removes existing records from a table.
- ALWAYS include a WHERE clause — without it every row is deleted.

SYNTAX

```
DELETE FROM table_name
WHERE condition;
```

EXAMPLE

```
DELETE FROM City
WHERE ID = 3285;
```

WHAT YOU'LL DO

Hands-On Labs — Data Processing and Analysis

Lab 4

- Query Data with SELECT

Lab 5

- Filter with SQL Operators

Lab 6

- Insert, Update and Delete Records

TOPIC 02 · HANDS-ON

Query Data with SELECT

LAB 4

Query the SG Mart product catalogue with SELECT: retrieve all columns, chosen columns, distinct values and filtered rows, then combine sorting and limits to answer real merchandising questions.

You'll build

A set of working queries answering questions about SG Mart's products, categories and outlets.

Tools: SQLite Studio, sgmart database, lab-04 dataset

Lab 4 — How It Flows

1. SELECT * and named columns
2. SELECT DISTINCT
3. Filter with WHERE
4. Sort & limit for top-N answers

LAB 4 · VERIFY

Query Data with SELECT

✅ Test it

Each query runs without error; the DISTINCT query returns 8 category codes; the top-10 price query is led by Pineapple Tarts 300g at $11.90; and the largest outlet is SG Mart Tampines Hub (1680.5 sqm).

TOPIC 02 · HANDS-ON

Filter with SQL Operators

LAB 5

Sharpen your WHERE clauses with AND/OR/NOT, IN, BETWEEN, LIKE wildcards, NULL tests and aliases to slice the SG Mart customer and staff data precisely.

You'll build

A query toolkit covering every SQL operator, answering pattern- and range-based questions on the SG Mart data.

Tools: SQLite Studio, sgmart database, lab-05 dataset

Lab 5 — How It Flows

1. AND / OR / NOT conditions
2. IN & BETWEEN ranges
3. LIKE pattern matching
4. IS NULL & aliases

LAB 5 · VERIFY

Filter with SQL Operators

✅ Test it

The IN query returns 16 Gold and Platinum members ordered by points; BETWEEN returns 11 products from $3.20 to $5.95; LIKE 'Frozen%' returns the 2 frozen lines; and IS NULL returns the 5 staff with no email, shown under the alias StaffName.

TOPIC 02 · HANDS-ON

Insert, Update and Delete Records

LAB 6

Change data safely: INSERT a new product line, UPDATE prices with a WHERE clause, DELETE a discontinued record, and reload the dataset from the provided seed script.

You'll build

A modified Products table proving you can add, change and remove records — plus a cleanly reloaded database.

Tools: SQLite Studio, sgmart database, lab-06 seed script

Lab 6 — How It Flows

1. INSERT a new record
2. Verify with SELECT
3. UPDATE with WHERE
4. DELETE with WHERE
5. Reload from the seed script

LAB 6 · VERIFY

Insert, Update and Delete Records

✅ Test it

The SKU1026 row appears after the INSERT and is gone after the DELETE; the three new members are added; Beverages prices rise by 10% (Sparkling Water 1.5L goes from 2.35 to 2.59); and after reloading, the counts return 25 products, 60 customers and 180 orders.

TOPIC RECAP

Recap — Data Processing and Analysis

- You can now: LO2 — apply data processing and analysis using SQL queries (A3, A5)
- You can now: LO2 — analyse data with operators, patterns and sorting (A3, A5)
- You can now: LO2 — process data by inserting, updating and deleting records (A3, A5)
- Full step-by-step instructions for every lab are in the Learner Guide.

Lunch Break

1:30 pm – 2:30 pm

03

TOPIC 03

Data Transformation

Aggregate Data · Join Data · Group Data

TSC MAPPING · A4, A6, K5, K6

Key Concepts — Data Transformation

1

Aggregate functions

COUNT, AVG and SUM collapse many rows into one summary value.

2

Joins

INNER, LEFT, RIGHT and FULL OUTER joins combine rows from two tables on a matching key.

3

Set operations

UNION stacks the result-sets of two SELECTs with matching columns.

4

Grouping

GROUP BY aggregates per group; HAVING filters groups the way WHERE filters rows.

Aggregate Functions

1

COUNT()
Number of rows matching a criterion.

2

AVG()
Average value of a numeric column.

3

SUM()
Total of a numeric column.

4

MIN()
Smallest value in a column.

5

MAX()
Largest value in a column.

6

With WHERE
Aggregate only the rows that match a filter.

COUNT · AVG · SUM

- Aggregate functions collapse many rows into a single summary value.
- Combine with WHERE to summarise a filtered subset.

SYNTAX

```
SELECT COUNT(column) FROM t;
SELECT AVG(column) FROM t;
SELECT SUM(column) FROM t;
```

EXAMPLE

```
SELECT COUNT(CountryCode) FROM City;

SELECT AVG(Population) FROM Country;

SELECT SUM(Population) FROM Country;
```

SQL Joins — Four Ways to Combine Tables

1

INNER JOIN

Only rows with matching values in BOTH tables.

2

LEFT JOIN

All rows from the left table + matches from the right (NULL if none).

3

RIGHT JOIN

All rows from the right table + matches from the left (NULL if none).

4

FULL OUTER JOIN

All rows from both tables, matched where possible.

INNER JOIN Keyword

- INNER JOIN selects records that have matching values in both tables.
- The ON clause names the matching key columns.

SYNTAX

```
SELECT columns
FROM table1
INNER JOIN table2
ON table1.col = table2.col;
```

EXAMPLE

```
SELECT City.Name, Country.Name
FROM City
INNER JOIN Country
ON City.CountryCode = Country.Code;
```

LEFT JOIN & RIGHT JOIN

- LEFT JOIN returns ALL rows of the left table and matched rows of the right — NULL where there is no match.
- RIGHT JOIN mirrors this for the right table. (SQLite supports LEFT JOIN; emulate RIGHT by swapping the tables.)

SYNTAX

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.col = table2.col;
```

EXAMPLE

```
SELECT c.CustomerName, o.OrderID
FROM Customers c
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID
ORDER BY c.CustomerName;
```

FULL OUTER JOIN & UNION

- FULL OUTER JOIN returns all records when there is a match in either table.
- UNION combines the result-sets of two SELECTs — same column count, types and order. In SQLite, a FULL JOIN is emulated with LEFT JOIN + UNION.

SYNTAX

```
SELECT cols FROM t1
FULL OUTER JOIN t2 ON t1.c = t2.c;

SELECT cols FROM t1
UNION
SELECT cols FROM t2;
```

EXAMPLE

```
SELECT l.*, r.* FROM left l
LEFT JOIN right r ON l.id = r.id
UNION
SELECT l.*, r.* FROM right r
LEFT JOIN left l ON l.id = r.id;
```

GROUP BY Statement

- GROUP BY collects rows sharing a value into summary rows — 'customers per country'.
- Used with COUNT, AVG, SUM, MIN, MAX to aggregate per group.

SYNTAX

```
SELECT col, AGG(col2)
FROM table
GROUP BY col
ORDER BY col;
```

EXAMPLE

```
SELECT CountryCode,
       AVG(Population)
FROM City
GROUP BY CountryCode;
```

HAVING Clause

- HAVING filters GROUPS — because WHERE cannot be used with aggregate functions.
- WHERE filters rows before grouping; HAVING filters the aggregated groups after.

SYNTAX

```
SELECT col, AGG(col2)
FROM table
GROUP BY col
HAVING AGG(col2) condition;
```

EXAMPLE

```
SELECT CountryCode, AVG(Population)
FROM City
GROUP BY CountryCode
HAVING AVG(Population) > 1000000;
```

WHAT YOU'LL DO

Hands-On Labs — Data Transformation

Lab 7

- Aggregate Data with COUNT, AVG and SUM

Lab 8

- Join Data from Multiple Tables

Lab 9

- Group Data with GROUP BY and HAVING

TOPIC 03 · HANDS-ON

Aggregate Data with COUNT, AVG and SUM

LAB 7

Summarise SG Mart's sales with COUNT, AVG, SUM, MIN and MAX: count transactions, average basket values, total revenue, and combine aggregates with DISTINCT and WHERE to answer real management questions.

You'll build

A management summary of SG Mart trading — order counts, average and total revenue, and filtered aggregate answers.

Tools: SQLite Studio, sgmart database, lab-07 dataset

Lab 7 — How It Flows

1. COUNT rows & distinct values
2. AVG a numeric column
3. SUM for totals
4. Aggregates with WHERE

LAB 7 · VERIFY

Aggregate Data with COUNT, AVG and SUM

✅ Test it

COUNT(*) returns 180 orders while COUNT(CustomerID) returns 163 (17 walk-ins are NULL); average line value is 13.48; total revenue is 9231.43; prices range from 2.10 to 11.90; and there are 5 distinct payment methods with 165 completed orders.

TOPIC 03 · HANDS-ON

Join Data from Multiple Tables

LAB 8

Combine tables: INNER JOIN orders to outlets and products, see how LEFT JOIN keeps the walk-in orders that have no member, chain three joins together, and emulate a FULL OUTER JOIN in SQLite using LEFT JOIN + UNION.

You'll build

Joined result-sets linking orders, outlets, customers and products, plus a full-outer-join of two practice tables showing matched and unmatched rows.

Tools: SQLite Studio, sgmart database, lab-08 dataset

Lab 8 — How It Flows

1. INNER JOIN two tables
2. Alias columns for readability
3. LEFT JOIN keeps unmatched rows
4. Chain three joins
5. UNION for a full outer join

LAB 8 · VERIFY

Join Data from Multiple Tables

✅ Test it

The inner joins return only matched rows; the LEFT JOIN lists every order with NULL names for walk-ins and the orphan count returns 17; the three-way join is led by SG Mart outlets selling Pineapple Tarts; and the full-join emulation returns ids 1–6 with NULLs on the sides that have no match.

TOPIC 03 · HANDS-ON

Group Data with GROUP BY and HAVING

LAB 9

Aggregate per group: revenue by outlet and by category with GROUP BY, then keep only the groups that matter with HAVING — and see why HAVING exists where WHERE cannot go.

You'll build

Grouped sales summaries of the SG Mart data — per-outlet and per-category totals, filtered to the groups that matter.

Tools: SQLite Studio, sgmart database, lab-09 dataset

Lab 9 — How It Flows

1. GROUP BY one column
2. Alias & ORDER the summary
3. HAVING filters groups
4. WHERE + GROUP BY + HAVING together

LAB 9 · VERIFY

Group Data with GROUP BY and HAVING

✅ Test it

The GROUP BY query returns one row per outlet (8 rows, led by OTL08 with 28 orders); revenue by outlet is led by SG Mart Serangoon NEX at 1496.82; the HAVING > 13 query returns the 5 qualifying outlets; and 18 members have 4 or more orders.

TOPIC RECAP

Recap — Data Transformation

- You can now: LO3 — transform data with aggregate functions (A4, A6)
- You can now: LO3 — transform data from multiple sources with joins and unions (A4, A6)
- You can now: LO3 — transform data by grouping and filtering groups (A4, A6)
- Full step-by-step instructions for every lab are in the Learner Guide.

Tea Break

15 minutes

04

TOPIC 04

Introduction to Data Warehouse

Overview of Data Warehouse · Relational Databases · Stored Procedures

TSC MAPPING · A7, A8, A9, K3, K7

Key Concepts — Introduction to Data Warehouse

1

Data warehouse

A consolidated, historical, read-mostly database kept separate from operational systems for analysis.

2

OLAP vs OLTP

OLAP serves complex analytical queries; OLTP serves fast day-to-day transactions.

3

RDBMS & E-R model

The relational model plus entity-relationship design maps business entities to tables and keys.

4

Stored procedures

Prepared SQL saved on the server (SQL Server) for reuse — callable with parameters via EXEC.

What is a Data Warehouse?

1

Separate store

Kept apart from the organisation's operational database.

2

Historical

Consolidated historical data with no frequent updates.

3

For decisions

Helps executives organise, understand and use data strategically.

4

Integrating

Merges diverse application systems into one analytical view.

TOPIC 04

Data Warehouse vs Operational Database

Operational DB (OLTP)

- Built for known tasks: search, index, update
  - Many concurrent short transactions
  - Current, constantly-changing data
  - Normalised for write efficiency

Data Warehouse (OLAP)

- Built for complex analytical queries
  - Read-mostly, batch loaded
  - Consolidated historical data
  - Structured for fast analysis

Data Warehouse Features

1

Subject oriented

Organised around subjects — product, customers, sales — not daily operations.

2

Integrated

Built by integrating data from heterogeneous sources into one schema.

3

Time variant

Data is identified with a time period, enabling trend analysis.

4

Non-volatile

Loaded data is not erased when new data is added.

Data Warehouse Applications

1. **Financial services**
   Portfolio and risk analysis.

2. **Banking**
   Customer and product profitability.

3. **Consumer goods**
   Sales and promotion analysis.

4. **Retail**
   Basket analysis and inventory.

5. **Manufacturing**
   Controlled process optimisation.

6. **Closed loop**
   Plan → execute → assess feedback for management.

OLAP Servers

1

ROLAP

Relational OLAP — analysis directly over relational tables.

2

MOLAP

Multidimensional OLAP — pre-computed data cubes.

3

HOLAP

Hybrid OLAP — relational detail with cube summaries.

4

Specialized SQL Servers

Engines tuned for analytical SQL workloads.

TOPIC 04

RDBMS — the Basis for SQL

What an RDBMS is

- Relational Database Management System
  - Data lives in tables of rows and columns
  - Basis of SQL Server, DB2, Oracle, MySQL, Access
  - Tables split into fields (columns) and records (rows)

Why it wins

- Simple yet powerful relational model
  - Tracks inventories, e-commerce, customer data
  - Fits any data where points relate to each other
  - Managed securely, consistently, at scale

DATA MAPPING

E-R Model → Database

1. Identify entities — the business objects (Routes, Timetables, Stations)
2. List attributes and pick each entity's primary key
3. Draw relationships — link entities with foreign keys
4. CREATE TABLE for each entity with PK & FK constraints
5. Load the data and verify with SELECT queries

TOPIC 04

Case Study: SMRT Public Transport

The business

- SMRT operates buses, taxis and trains across Singapore
  - Needs schedules, routes and disruption history
  - Analyses service impact and severity

The E-R model

- Routes — route_id (PK), type, code, name, direction
  - Timetables — timetable_id (PK), route_id (FK), station_code (FK), frequency, first/last trip
  - DisruptedRoutes — disrupt_no (PK), date, type, details, timetable_id (FK)

Stored Procedures (SQL Server)

- A stored procedure is prepared SQL saved on the server, reusable by name.
- Not available in SQLite — this is SQL Server / MySQL territory.

SYNTAX

```
CREATE PROCEDURE procedure_name
AS
sql_statement
GO;

EXEC procedure_name;
```

EXAMPLE

```
CREATE PROCEDURE SelectAllCustomers
AS
SELECT * FROM Customers
GO;

EXEC SelectAllCustomers;
```

Stored Procedure with Parameters

- Parameters let one procedure act on the values passed to it.
- Call EXEC with the parameter values.

SYNTAX

```
CREATE PROCEDURE name @param type
AS
sql_statement_using @param
GO;

EXEC name @param = value;
```

EXAMPLE

```
CREATE PROCEDURE SelectAllCustomers
  @City nvarchar(30)
AS
SELECT * FROM Customers
WHERE City = @City
GO;

EXEC SelectAllCustomers
  @City = 'London';
```

WHAT YOU'LL DO

Hands-On Labs — Introduction to Data Warehouse

Lab 10

- Map an E-R Model to Database Tables (SMRT Case Study)

Lab 11

- Author Stored Procedures for SQL Server

TOPIC 04 · HANDS-ON

Map an E-R Model to Database Tables (SMRT Case Study)

LAB 10

Take the SMRT public-transport E-R model (Routes, Stations, Timetables, DisruptedRoutes) and map it to physical tables: create each entity with its primary key, wire the foreign-key relationships, load the mock operational data and verify the model with joined queries.

You'll build

A four-table transport schema in SQLite that faithfully implements the SMRT E-R model with PK/FK mappings, loaded with real-shaped route, timetable and disruption data.

Tools: SQLite Studio, sgmart_practice database, lab-10 dataset (Routes, Stations, Timetables, DisruptedRoutes)

Lab 10 — How It Flows

1. Review the SMRT E-R model
2. CREATE Routes (parent)
3. CREATE Stations & Timetables (children)
4. CREATE DisruptedRoutes
5. Load data & verify with joins

LAB 10 · VERIFY

Map an E-R Model to Database Tables (SMRT Case Study)

✅ Test it

The four tables exist with their PK/FK constraints visible in the DDL; Routes returns 8 rows; the NSL station query returns 4 stations in line order; the three-way join returns the 6 disruptions each linked to a route and station; and the LEFT JOIN summary lists every route including those with zero disruptions.

TOPIC 04 · HANDS-ON

Author Stored Procedures for SQL Server

LAB 11

Write stored procedures in SQL Server syntax against the SG Mart data — a plain procedure and a parameterised one — and understand where they run (SQL Server / MySQL, not SQLite) and why warehouses use them for repeatable batch work.

You'll build

Stored-procedure scripts (plain and parameterised) ready to run on SQL Server, plus the EXEC calls that invoke them.

Tools: SQL Editor, SQL Server syntax (sqliteonline.com's MS SQL engine, or any SQL Server), lab-11 dataset (seed_mysql.sql)

Lab 11 — How It Flows

1. Set up the tables on SQL Server
2. Write CREATE PROCEDURE
3. EXEC the procedure
4. Add a parameter
5. EXEC with a value

LAB 11 · VERIFY

Author Stored Procedures for SQL Server

✅ Test it

All three procedures create without error; EXEC SelectAllCustomers returns the full 60-member list; EXEC SelectCustomersByTier @Tier = 'Platinum' returns only the 7 Platinum members ordered by points; and the two-parameter procedure returns one summary row for OTL08.

TOPIC RECAP

Recap — Introduction to Data Warehouse

- You can now: LO4 — apply data mapping from an E-R model to a data warehouse schema (A7, A8, A9)
- You can now: LO4 — operate data-warehouse platform capabilities with reusable stored procedures (A7, A8, A9)
- Full step-by-step instructions for every lab are in the Learner Guide.

WRAP-UP

Course Summary & Next Steps

LEARNING OUTCOMES

What You Achieved

1

Data Modeling

Created databases, tables, constraints, keys and relationships in SQLite Studio.

2

Data Processing & Analysis

Queried, filtered, sorted, inserted, updated and deleted data with SQL.

3

Data Transformation

Aggregated, joined, unioned and grouped data across multiple tables.

4

Data Warehouse

Mapped an E-R model to tables and authored stored procedures.

CONTINUE YOUR JOURNEY · tertiarycourses.com.sg

Recommended Next Courses

1. **Building Professional Websites with WordPress**
   WSQ · Tertiary Infotech

2. **Building a Successful eCommerce Store with WooCommerce**
   WSQ · Tertiary Infotech

3. **Mastering HTML5 and CSS3 Fundamentals for Professional Web Design**
   WSQ · Tertiary Infotech

4. **Hands-On Web App Development with Javascript**
   WSQ · Tertiary Infotech

5. **Running a Successful eCommerce Store with Shopify**
   WSQ · Tertiary Infotech

ENQUIRIES DURING AND AFTER THE CLASS

Support

1

Email

enquiry@tertiaryinfotech.com

2

Telephone

+65 6100 0613

3

Website

www.tertiarycourses.com.sg

4

LMS / TMS

lms-tms.tertiaryinfotech.com

COURSE FEEDBACK & CERTIFICATION

Certificate & TRAQOM Survey (Mandatory)

1. Log in to lms-tms.tertiaryinfotech.com
2. Complete the TRAQOM course feedback survey — mandatory for WSQ funding
3. Be assessed as Competent in the PP and OQ
4. Download your e-certificate from the LMS

ASSESSMENT · 90 MINUTES TOTAL

Final Assessment

1

Practical Performance (PP)

Hands-on SQL tasks · 70 minutes · open book.

2

Oral Questioning (OQ)

Spoken knowledge questions · 20 minutes · one-to-one.

3

Before you begin

Take the Assessment digital attendance (TRAQOM).

4

After the OQ

Submit your work on the LMS, then sign the Assessment Summary Record.

ON ASSESSMENT DAY

Assessment Flow

1. TRAQOM survey — scan the QR code on the LMS
2. Assessment digital attendance — scan the SSG QR
3. Practical Performance (PP) — 70 min, open book
4. Oral Questioning (OQ) — 20 min, 1-to-1
5. Submit on the LMS — upload your answers
6. Sign the Assessment Summary Record

Digital Attendance (Mandatory)

1

Three times a day

Take the AM, PM and Assessment digital attendance — mandatory for WSQ-funded courses.

2

QR code from SSG

The trainer or administrator displays the attendance QR code from the SSG portal.

3

Scan and submit

Scan the QR code with your mobile phone camera and submit your attendance.

4

75% minimum

A minimum 75% attendance rate is required to be eligible for assessment and funding.

SEE YOU IN THE NEXT COURSE

Thank You!

You can now model, query, transform and warehouse data with SQL — happy querying!